Richard Smith, Andrew Hunter
2017-11-27
Audience: Core, Library

P0722R2: Efficient sized delete for variable sized classes

This paper provides wording for the feature described by P0722R1.

Append to [basic.stc.dynamic.deallocation] (6.7.4.2) paragraph 1

> […] or declared in global scope. **A deallocation function with at least two parameters whose second parameter type is of type `std::destroying_delete_t` is a *destroying operator delete*. A destroying operator delete shall be a class member function named `operator delete`.**

Change in [basic.stc.dynamic.deallocation] (6.7.4.2) paragraph 2:

> Each deallocation function shall return `void`. **If the function is a destroying operator delete declared in class type `C`, the type of its first parameter shall be `C*`; otherwise, the type of** ~~and~~ its first parameter shall be `void*`. A deallocation function may have more than one parameter. A *usual deallocation function* is a deallocation function ~~that has:~~ **whose parameters after the first are**
>
> - ~~exactly one parameter; or~~ **optionally, a parameter of type `std::destroying_delete_t`, then**
> - ~~exactly two parameters, the type of the second being either `std::align_val_t` or~~ **optionally, a parameter of type `std::size_t`**[Footnote: The global `operator delete(void*, std::size_t)` precludes use of an allocation function `void operator new(std::size_t, std::size_t)` as a placement allocation function]~~; or~~**, then**
> - ~~exactly three parameters, the type of the second being `std::size_t` and the type of the third being~~ **optionally, a parameter of type** `std::align_val_t`.
>
> **A destroying operator delete shall be a usual deallocation function.** A deallocation function may be an instance of a function template. Neither the first parameter nor the return type shall depend on a template parameter. [Note: That is, a deallocation function template shall have a first parameter of type `void*` and a return type of `void` (as specified above). — end note] A deallocation function template shall have two or more function parameters. A template instance is never a usual deallocation function, regardless of its signature.

Change in [expr.delete] (8.3.5) paragraph 3:

> In the first alternative (delete object), if the static type of the object to be deleted is different from its dynamic type, **and the selected deallocation function (see below) is not a destroying operator delete,** the static type shall be a base class of the dynamic type of the object to be deleted and the static type shall have a virtual destructor or the behavior is undefined. In the second alternative (delete array) if the dynamic type of the object to be deleted differs from its static type, the behavior is undefined.

Change in [expr.delete] (8.3.5) paragraph 8:

> If the value of the operand of the *delete-expression* is not a null pointer value, **and the selected deallocation function (see below) is not a destroying operator delete,** the *delete-expression*

will invoke the destructor (if any) for the object or the elements of the array being deleted. In the case of an array, the elements will be destroyed in order of decreasing address (that is, in reverse order of the completion of their constructor; see 15.6.2).

Change in [expr.delete] (8.3.5) paragraph 10:

If deallocation function lookup finds more than one usual deallocation function, the function to be called is selected as follows:

- **If any of the deallocation functions is a destroying operator delete, all deallocation functions that are not destroying operator deletes are eliminated from further consideration.**
- If the type has new-extended alignment, a function with a parameter of type `std::align_val_t` is preferred; otherwise a function without such a parameter is preferred. If **any** ~~exactly one preferred function is found, that function is selected and the selection process terminates. If more than one~~ preferred function**s are** ~~is~~ found, all non-preferred functions are eliminated from further consideration.
- **If exactly one function remains, that function is selected and the selection process terminates.**
- If the deallocation functions have class scope, the one without a parameter of type `std::size_t` is selected.
- If the type is complete and if, for the second alternative (delete array) only, the operand is a pointer to a class type with a non-trivial destructor or a (possibly multi-dimensional) array thereof, the function with a parameter of type `std::size_t` is selected.
- Otherwise, it is unspecified whether a deallocation function with a parameter of type `std::size_t` is selected.

Drafting note: the pre-existing wording makes the incorrect assumption that (after the alignment step) there must be one sized operator delete and one non-sized operator delete. That appears to be false: we could have (e.g.) both `operator delete(void*)` and `operator delete(void*, ...)` here (both are usual deallocation functions).

Change in [expr.delete] (8.3.5) paragraph 11:

For the delete object case, the deleted object is the object denoted by the operand if its static type does not have a virtual destructor, and its most-derived object otherwise. [Note: If the deallocation function is not a destroying operator delete and the deleted object is not the most derived object in the former case, the behavior is undefined, as stated above. —] For the delete array case, the deleted object is the array object. When a delete-expression is executed, the selected deallocation function shall be called with the address of the ~~most-derived~~ **deleted** object in the delete object case, or the address of the **deleted** object suitably adjusted for the array allocation overhead (8.3.4) in the delete array case, as its first argument. **If a destroying operator delete is used, an unspecified value is passed as the argument corresponding to the parameter of type `std::destroying_delete_t`.** If a deallocation function with a parameter of type `std::align_val_t` is used, the alignment of the type of the **deleted** object ~~to be deleted~~ is passed as the corresponding argument. If a deallocation function with a parameter of type `std::size_t` is used, the size of the ~~most-derived type~~ **deleted object in the delete object case**, or of the array plus allocation overhead~~, respectively~~ **in the delete array case**, is passed as the corresponding argument. ~~[Footnote: If the static type of the object to be deleted is complete and is different from the dynamic type, and the destructor is not virtual, the size might be incorrect, but that case is already undefined, as stated above.]~~ [Note: If this results in a call to a ~~usual~~ **replaceable** deallocation function, and either the first argument was not the result of a prior call to a ~~usual~~ **replaceable** allocation function or the second **or third** argument was not the corresponding argument in said call, the behavior is undefined (21.6.2.1, 21.6.2.2). — end note]

Change in [expr.delete] (8.3.5) paragraph 12:

> Access and ambiguity control are done for ~~both~~ the deallocation function and**, if the deallocation function is not a destroying operator delete or the destructor is virtual, for** the destructor (15.4, 15.5).

Change in [class.free] (15.5) paragraph 4:

> Class-specific deallocation function lookup is a part of general deallocation function lookup (8.3.5) and occurs as follows. If the delete-expression is used to deallocate a class object whose static type has a virtual destructor, the deallocation function is the one selected at the point of definition of the dynamic type's virtual destructor (15.4).[Footnote] Otherwise, if the delete-expression is used to deallocate an object of class T or array thereof, ~~the static and dynamic types of the object shall be identical and~~ the deallocation function's name is looked up in the scope of T. If this lookup fails to find the name, general deallocation function lookup (8.3.5) continues. If the result of the lookup is ambiguous or inaccessible, or if the lookup selects a placement deallocation function, the program is ill-formed.

Change in [class.free] (15.5) paragraph 6:

> Since member allocation and deallocation functions are static they cannot be virtual. [Note: However, when the cast-expression of a delete-expression refers to an object of class type, because the deallocation function actually called is looked up in the scope of the class that is the dynamic type of the object~~,~~ if the destructor is virtual, the effect is the same **in that case**. For example,
>
> ```
> struct B {
>   virtual ~B();
>   void operator delete(void*, std::size_t);
> };
> struct D : B {
>   void operator delete(void*);
> };
> struct E : B {
>   void operator delete(void*, std::destroying_delete_t);
> };
> void f() {
>   B* bp = new D;
>   delete bp; // 1: uses D::operator delete(void*)
>   bp = new E;
>   delete bp; // 2: uses E::operator delete(void*, std::destroying_delete_t)
> }
> ```
>
> Here, storage for the ~~non-array~~ object of class `D` is deallocated by `D::operator delete()`, **and the object of class `E` is destroyed and its storage is deallocated by `E::operator delete()`, due to the virtual destructor. — end note] [Note: …]**

Change in header <new> synopsis in [new.syn] (21.6.1):

```
namespace std {
  class bad_alloc;
  class bad_array_new_length;

  // [new.destr.del], destroying delete indicator
  struct destroying_delete_t { see below };
  inline constexpr destroying_delete_t destroying_delete(unspecified);
```

```
  struct nothrow_t { explicit nothrow_t() = default; };
  […]
```

Add a new subclause before the existing 21.6.4 [ptr.launder]:

Destroying delete indicator [new.destr.del]

```
struct destroying_delete_t { see below };
inline constexpr destroying_delete_t destroying_delete(unspecified);
```

The struct `destroying_delete_t` is an empty structure type used as a unique type to indicate that a deallocation function is a destroying operator delete ([basic.stc.dynamic.deallocation]).

Type `destroying_delete_t` shall not have a default constructor or an initializer-list constructor, and shall not be an aggregate.